Mentimeter

Haskell: Tipuri de date utilizator

Mentimeter

Limbaj puternic tipat

- conversiile nu se fac automat, trebuie facute **explicit** (ex: fromIntegral pentru conversia de la Int la Double)
- consecinta: toate elementele unei liste trebuie sa aiba acelasi tip

Mentimeter

Limbaj tipat static (la compilare)

- sinteza de tip: verificam tipurile tuturor expresiilor de la compilare
- pot sa mai apara si erori dinamice (ex: head pe o lista empty)

Mentimeter

```haskell
type Tuple = (Int, Int, Int)

-- tipul lui tuple1 este detectat ca:
--        tuple1 :: (Integer, Integer, Integer)
tuple1 = (1, 2, 3)

tuple2 :: Tuple
tuple2 = (1, 2, 3)
```

Type (~typedef)

Mentimeter

Explicatii

- daca nu declar si tipul variabilei atunci Haskell nu o asociaza automat cu tipul declarat
- **cel mai bine: puneti si tipul pentru a se face verificarea de tip**
- ex: Tuple = (Int, Int, Int), daca primeste (1.0, 2, 3) o sa apara o eroare pentru ca 1.0 nu este de tip Int

Mentimeter

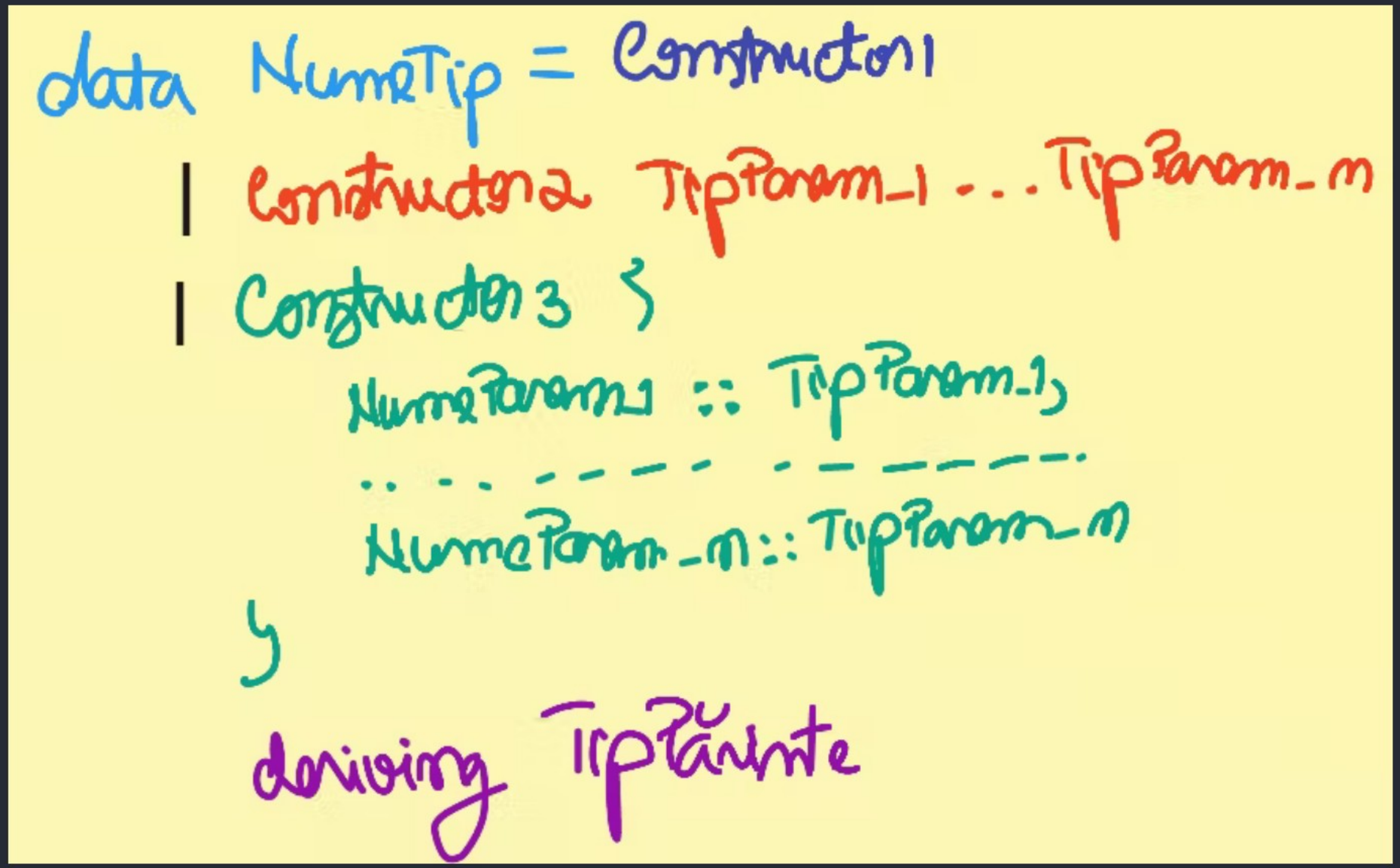
```
10  type Node = (Node, Int, Node)
11
```

Command Prompt - stack exec ghci tipuri-doc.hs

```
tipuri-doc.hs:10:1: error:
    Cycle in type synonym declarations:
      tipuri-doc.hs:10:1-29: type Node = (Node, Int, Node)
   |
10 | type Node = (Node, Int, Node)
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Nu pot sa am tipuri recursive definite cu type



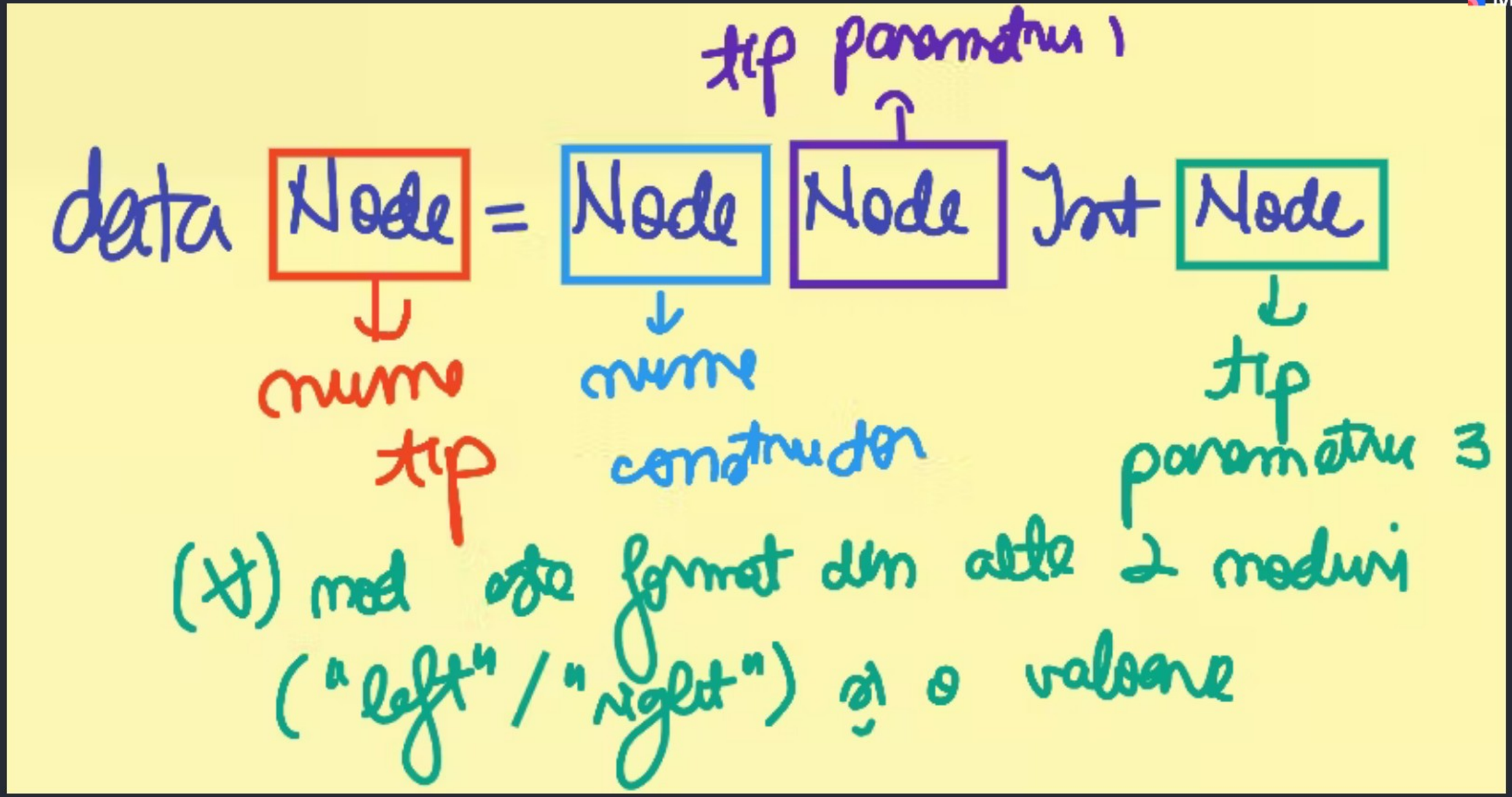

data NumeTip = Constructor1
| Constructor2 TipParam_1 ... TipParam_n
| Constructor3 {
NumeParam_1 :: TipParam_1
NumeParam_n :: TipParam_n
}
deriving TipClasa

Data

Mentimeter

Constructia data

- permite definirea tipurilor de date algebrice
- pot sa am constructori cu/fara parametri
- poate mosteni o clasa (deriving)



Constructorii pot sa aiba acelasi nume cu tipul, dar hai sa nu **cateodata**

Mentimeter

```
data MyStudent = Anonymous
       | NoName Int
       | MyStudent String Int
```

Command Prompt - stack exec ghci tipuri-doc.hs

```
*Main> :t Anonymous
Anonymous :: MyStudent
*Main> :t NoName
NoName :: Int -> MyStudent
*Main> :t MyStudent
MyStudent :: String -> Int -> MyStudent
```

Orice constructor este o functie

Mentimeter

```haskell
data Day = Monday | Tuesday | End
data Month = January | February
data Year = Year Int

data Data = Nada
          | OnlyDay Day
          | OnlyMonth Month
          | OnlyYear Year
          | AllIn Day Month Year
```

Tipuri enumerate

Mentimeter

Explicatii

- pot sa am mai multi constructori si ii inlantui cu |
- ex: Month are 2 constructori fara parametri (January/February) si ii pot folosi pe amandoipo
- ex2: pot crea date cu oricare constructor definit

Mentimeter

Cum imi dau seama ce constructor a fost folosit pentru crearea unui obiect?

- in functie de constructor am acces la date diferite (ex: numai la luna, an, etc)
- trebuie sa stiu constructorul folosit ca sa stiu ce date am
- **solutie: pattern matching**

Mentimeter

```haskell
import Text.Printf

data Data = Nada
    | OnlyDay String
    | OnlyMonth String
    | OnlyYear Int
    | AllIn String String Int

printData :: Data -> String
printData (OnlyDay day) = printf "Day %s" day
printData (OnlyMonth month) = printf "Month %s" month
printData (OnlyYear year) = printf "Year %d" year
printData (AllIn day month year) = printf "Day %s, month %s, year %d" day month year
```

Exemplu pattern matching

Mentimeter

Tipuri inregistrate

- asociem nume campurilor structurii de date
- se genereaza automat un fel de getters

Mentimeter

```haskell
data MyStudent = Anoymous
    | MyStudent {
        name :: String,
        age :: String
    }
```

Command Prompt - stack exec ghci tipuri-doc.hs

```
*Main> :t name
name :: MyStudent -> String
*Main> :t age
age :: MyStudent -> String
```

Exemplu

Mentimeter

Explicatii

- pentru fiecare tip inregistrat se genereaza o functie cu acelasi nume care intoarce campul
- ce fac daca apelez name Anonymous? **eroare dinamica**
- pot sa combin constructori cu tipuri inregistrate cu alte tipuri de constructori, dar functiile astea nu merg pe toti

Mentimeter

```haskell
data MyStudent = Anoymous
    | MyStudent {
        name :: String,
        age :: Int
    }

-- isi da seama singur de campuri in
-- functie de tipul si ordinea lor
student1 = MyStudent "Ana" 20

student2 = MyStudent {
    name = "ana",
    age = 20
}
```

Crearea unui student

Mentimeter

```haskell
data Pair1 a = Pair1 a a
data Pair2 a b = Pair2 a b
```

Command Prompt - stack exec ghci tipuri-doc.hs

```
*Main> p1 = Pair1 1 1
*Main> p1 = Pair1 1 'a'

<interactive>:9:12: error:
    * No instance for (Num Char) arising from the literal `1'
    * In the first argument of `Pair1', namely `1'
      In the expression: Pair1 1 'a'
      In an equation for `p1': p1 = Pair1 1 'a'
*Main> p2 = Pair2 1 'a'
*Main> p2 = Pair2 1 1
```

Tipuri parametrizate

Mentimeter

Explicatii

- putem folosi tipuri generice (ex: a, b) pentru definirea altor tipuri (ex: Pair1, Pair2)
- Pair1 primeste 2 parametri ambii de acelasi tip a (**nu pot face perechi cu tipuri diferite**)
- Pair2 primeste 2 parametri cu tipurile a si b (tipurile pot fi diferite sau nu)

Mentimeter

```
data List a = Empty | Cons a (List a)
data TOptional a = TNothing | TJust a
```

Alte exemple

Mentimeter

```haskell
data TList a = EmptyList | Cons a (TList a) deriving Show
data TTree a = EmptyTree | TreeNode (TTree a) a (TTree a) deriving Show
data TNatural = Zero | Succ TNatural deriving Show
```

Tipuri recursive

Mentimeter

newtype

- asemanator data dar cu un singur constructor cu un singur parametru
- **pot folosi perechi pentru a-i da mult de un parametru**
- ex: newtype Tuple a = Tuple (a, a, a)

Mentimeter

```haskell
data Tree = Null | Node Tree Tree Int deriving Show

search Null x = Null
search tree@(Node left right value) x
    | x == value = tree
    | x < value = search left x
    | otherwise = search right x
```

Alias-uri